

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

Oleh:

Randy Febrian

NIM. 2310817110013

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to The Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Randy Febrian
NIM : 2310817110013

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	7
B. Output Program	36
C. Pembahasan	39
TAUTAN GIT	51

DAFTAR GAMBAR

Gambar 1. Screenshot Home Screen Soal 1	36
Gambar 2. Screenshot Detail Screen Soal 1	37
Gambar 3. Screenshot Intent Browser Soal 1	38

DAFTAR TABEL

Tabel 1. Jawaban Soal 1 Ext.kt	7
Tabel 2. Jawaban Soal 1 GenreConstant.kt	7
Tabel 3. Jawaban Soal 1 K.kt	8
Tabel 4. Jawaban Soal 1 Response.kt.....	8
Tabel 5. Jawaban Soal 1 ApiMapper.kt.....	8
Tabel 6. Jawaban Soal 1 MovieDetailMapper.kt	8
Tabel 7. Jawaban Soal 1 MovieDao.kt	9
Tabel 8. Jawaban Soal 1 MovieDatabase.kt.....	9
Tabel 9. Jawaban Soal 1 MovieEntity.kt.....	10
Tabel 10. Jawaban Soal 1 MovieLocalMapper.kt	10
Tabel 11. Jawaban Soal 1 MovieApiMapperImpl.kt.....	11
Tabel 12. Jawaban Soal 1 MovieDetailMapperImpl.kt.....	13
Tabel 13. Jawaban Soal 1 MovieApiService.kt.....	13
Tabel 14. Jawaban Soal 1 MovieDto.kt.....	14
Tabel 15. Jawaban Soal 1 MovieDetailDto.kt.....	14
Tabel 16. Jawaban Soal 1 MovieRepositoryImpl.kt	16
Tabel 17. Jawaban Soal 1 DatabaseModule.kt	17
Tabel 18. Jawaban Soal 1 MovieModule.kt	18
Tabel 19. Jawaban Soal 1 Movie.kt.....	20
Tabel 20. Jawaban Soal 1 MovieDetail.kt.....	20
Tabel 21. Jawaban Soal 1 MovieRepository.kt	21
Tabel 22. Jawaban Soal 1 DetailScreenViewModel.kt	21
Tabel 23. Jawaban Soal 1 HomeViewModel.kt	22
Tabel 24. Jawaban Soal 1 HomeScreen.kt	23
Tabel 25. Jawaban Soal 1 DetailScreen.kt	28
Tabel 26. Jawaban Soal 1 AppNavigation.kt	34
Tabel 27. Jawaban Soal 1 MovieApplication.kt.....	35
Tabel 28. Jawaban Soal 1 MainActivity.kt.....	35

SOAL 1

Soal Praktikum

1. Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b. Gunakan KotlinX Serialization sebagai library JSON.
- c. Gunakan library seperti Coil atau Glide untuk image loading.
- d. API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- e. Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
- f. Gunakan caching strategy pada Room..
- g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

Tabel 1. Jawaban Soal 1 Ext.kt

```

1 package com.example.retrofit.utils
2
3 import android.util.Log
4 import kotlinx.coroutines.flow.Flow
5
6 suspend fun <T> Flow<Response<T>>.collectAndHandle(
7     onError: (Throwable) -> Unit = { throwable ->
8         Log.e("collectAndHandle", "Error: ", throwable)
9     },
10    onLoading: () -> Unit = {},
11    stateReducer: (T) -> Unit,
12 ) {
13     collect { response ->
14         when (response) {
15             is Response.Loading -> onLoading()
16             is Response.Success ->
17 stateReducer(response.data)
18             is Response.Error -> {
19                 val error = response.error ?:
20 Exception("Unknown error occurred")
21                 onError(error)
22             }
23         }
24     }
25 }

```

Tabel 2. Jawaban Soal 1 GenreConstant.kt

```

1 package com.example.retrofit.utils
2
3 object GenreConstants {
4     private val genreMap = mapOf(
5         28 to "Action",
6         12 to "Adventure",
7         16 to "Animation",
8         35 to "Comedy",
9         80 to "Crime",
10        99 to "Documentary",
11        18 to "Drama",
12        10751 to "Family",
13        14 to "Fantasy",
14        36 to "History",
15        27 to "Horror",
16        10402 to "Music",
17        9648 to "Mystery",

```

```

18         10749 to "Romance",
19         878 to "Science Fiction",
20         10770 to "TV Movie",
21         53 to "Thriller",
22         10752 to "War",
23         37 to "Western"
24     )
25
26     fun getGenreNameById(id: Int): String {
27         return genreMap[id] ?: "Unknown"
28     }
29 }

```

Tabel 3. Jawaban Soal 1 K.kt

```

1 package com.example.retrofit.utils
2
3 object K {
4     const val BASE_URL =
5     "https://api.themoviedb.org/3/"
6     const val BASE_IMAGE_URL =
7     "https://image.tmdb.org/t/p/w500/"
8     const val MOVIE_ENDPOINT = "discover/movie"
9 }

```

Tabel 4. Jawaban Soal 1 Response.kt

```

1 package com.example.retrofit.utils
2
3 sealed class Response<T>{
4     class Success<T>(val data:T): Response<T>()
5     class Error<T>(val error:Throwable?,val data:T? =
6     null): Response<T>()
7     class Loading<T>: Response<T>()
8 }

```

Tabel 5. Jawaban Soal 1 ApiMapper.kt

```

1 package com.example.retrofit.common.data
2
3 interface ApiMapper<Domain, Entity> {
4     fun mapToDomain(apiDto: Entity): Domain
5 }

```

Tabel 6. Jawaban Soal 1 MovieDetailMapper.kt

```

1 package com.example.retrofit.common.data
2
3 import

```

4	com.example.retrofit.data.remote.models.MovieDetailDto
5	import com.example.retrofit.domain.models.MovieDetail
6	
7	interface MovieDetailMapper {
8	fun mapMovieDetailDtoToDomain(dto: MovieDetailDto):
9	MovieDetail
10	}

Tabel 7. Jawaban Soal 1 MovieDao.kt

1	package com.example.retrofit.data.local.dao
2	
3	import androidx.room.*
4	import
5	com.example.retrofit.data.local.entities.MovieEntity
6	import
7	com.example.retrofit.data.local.entities.MovieDetailEnt
8	ity
9	import kotlinx.coroutines.flow.Flow
10	
11	@Dao
12	interface MovieDao {
13	
14	@Query("SELECT * FROM movies ORDER BY popularity
15	DESC")
16	suspend fun getAllMovies(): List<MovieEntity>
17	
18	@Query("SELECT * FROM movies ORDER BY popularity
19	DESC")
20	fun getAllMoviesFlow(): Flow<List<MovieEntity>>
21	
22	@Insert(onConflict = OnConflictStrategy.REPLACE)
23	suspend fun insertMovies(movies: List<MovieEntity>)
24	
25	@Query("DELETE FROM movies")
26	suspend fun clearAll()
27	
28	@Query("SELECT COUNT(*) FROM movies")
29	suspend fun getMoviesCount(): Int
30	}

Tabel 8. Jawaban Soal 1 MovieDatabase.kt

1	package com.example.retrofit.data.local.db
2	
3	import androidx.room.Database
4	import androidx.room.RoomDatabase

```

5 import com.example.retrofit.data.local.dao.MovieDao
6
7 @Database(entities =
8 [com.example.retrofit.data.local.entities.MovieEntity::
9 class], version = 2)
10 abstract class MovieDatabase : RoomDatabase() {
11     abstract fun movieDao(): MovieDao
12 }

```

Tabel 9. Jawaban Soal 1 MovieEntity.kt

```

1 package com.example.retrofit.data.local.entities
2
3 import androidx.room.Entity
4 import androidx.room.PrimaryKey
5
6 @Entity(tableName = "movies")
7 data class MovieEntity(
8     @PrimaryKey val id: Int,
9     val title: String,
10    val overview: String,
11    val releaseDate: String,
12    val voteAverage: Double,
13    val posterPath: String,
14    val backdropPath: String,
15    val originalLanguage: String,
16    val originalTitle: String,
17    val popularity: Double,
18    val video: Boolean,
19    val voteCount: Int,
20    val genreIds: String,
21 )

```

Tabel 10. Jawaban Soal 1 MovieLocalMapper.kt

```

1 package com.example.retrofit.data.local.mapper
2
3 import
4 com.example.retrofit.data.local.entities.MovieEntity
5 import com.example.retrofit.domain.models.Movie
6
7 fun MovieEntity.toDomain(): Movie {
8     return Movie(
9         id = id,
10        title = title,
11        overview = overview,
12        releaseDate = releaseDate,

```

```

13         voteAverage = voteAverage,
14         posterPath = posterPath,
15         backdropPath = backdropPath,
16         originalLanguage = originalLanguage,
17         originalTitle = originalTitle,
18         popularity = popularity,
19         video = video,
20         voteCount = voteCount,
21         genreIds = genreIds
22     )
23 }
24
25 fun Movie.toEntity(): MovieEntity {
26     return MovieEntity(
27         id = id,
28         title = title,
29         overview = overview,
30         releaseDate = releaseDate,
31         voteAverage = voteAverage,
32         posterPath = posterPath,
33         backdropPath = backdropPath,
34         originalLanguage = originalLanguage,
35         originalTitle = originalTitle,
36         popularity = popularity,
37         video = video,
38         voteCount = voteCount,
39         genreIds = genreIds
40     )
41 }

```

Tabel 11. Jawaban Soal 1 MovieApiMapperImpl.kt

```

1 package com.example.retrofit.data.mapper_impl
2
3 import com.example.retrofit.common.data.ApiMapper
4 import com.example.retrofit.data.remote.models.MovieDto
5 import com.example.retrofit.domain.models.Movie
6 import com.example.retrofit.utils.GenreConstants
7
8 class MovieApiMapperImpl : ApiMapper<List<Movie>,
9 MovieDto> {
10     override fun mapToDomain(apiDto: MovieDto):
11 List<Movie> {
12         return apiDto.results?.map { result ->
13             Movie(
14                 backdropPath =
15 formatEmptyValue(result?.backdropPath),

```

```

16         genreIds =
17 formatGenre(result?.genreIds),
18         id = result?.id ?: 0,
19         originalLanguage =
20 formatEmptyValue(result?.originalLanguage, default =
21 "language"),
22         originalTitle =
23 formatEmptyValue(result?.originalTitle, default =
24 "title"),
25         overview =
26 formatEmptyValue(result?.overview, default =
27 "overview"),
28         popularity = result?.popularity ?: 0.0,
29         posterPath =
30 formatEmptyValue(result?.posterPath),
31         releaseDate =
32 formatEmptyValue(result?.releaseDate, default =
33 "date"),
34         title = formatEmptyValue(result?.title,
35 default = "title"),
36         video = result?.video ?: false,
37         voteAverage = result?.voteAverage ?:
38 0.0,
39         voteCount = result?.voteCount ?: 0
40     )
41     } ?: emptyList()
42 }
43
44 private fun formatEmptyValue(value: String?,
45 default: String = ""): String {
46     if (value.isNullOrEmpty()) return "Unknown
47 $default"
48     return value
49 }
50
51 private fun formatGenre(genreIds: List<Int?>?):
52 String {
53     return genreIds?.mapNotNull { id ->
54         id?.let {
55 GenreConstants.getGenreNameById(it) }
56     }?.joinToString(", ") ?: "Unknown Genre"
57 }
58 }

```

Tabel 12. Jawaban Soal 1 MovieDetailMapperImpl.kt

```

1 package com.example.retrofit.data.mapper_impl
2
3 import
4 com.example.retrofit.common.data.MovieDetailMapper
5 import
6 com.example.retrofit.data.remote.models.MovieDetailDto
7 import com.example.retrofit.domain.models.MovieDetail
8
9 class MovieDetailMapperImpl : MovieDetailMapper {
10     override fun mapMovieDetailDtoToDomain(dto:
11 MovieDetailDto): MovieDetail {
12         return MovieDetail(
13             id = dto.id ?: -1,
14             title = dto.title ?: "",
15             overview = dto.overview ?: "",
16             posterPath = dto.posterPath ?: "",
17             releaseDate = dto.releaseDate ?: "",
18             genres = dto.genres?.map { it?.name ?:
19 "Unknown" } ?: listOf("Unknown"),
20             voteAverage = dto.voteAverage ?: 0.0,
21             voteCount = dto.voteCount ?: 0,
22             popularity = dto.popularity ?: 0.0,
23             backdropPath = dto.backdropPath ?: "",
24             originalLanguage = dto.originalLanguage ?:
25 "Unknown",
26             originalTitle = dto.originalTitle ?: "",
27             video = dto.video ?: false
28         )
29     }
30 }

```

Tabel 13. Jawaban Soal 1 MovieApiService.kt

```

1 package com.example.retrofit.data.remote.api
2
3 import com.example.retrofit.BuildConfig
4 import
5 com.example.retrofit.data.remote.models.MovieDetailDto
6 import com.example.retrofit.data.remote.models.MovieDto
7 import com.example.retrofit.utils.K
8 import retrofit2.http.GET
9 import retrofit2.http.Path
10 import retrofit2.http.Query
11
12 interface MovieApiService {
13

```

```

14     @GET(K.MOVIE_ENDPOINT)
15     suspend fun fetchDiscoverMovie(
16         @Query("api_key") apiKey: String =
17     BuildConfig.apiKey,
18         @Query("include_adult") includeAdult: Boolean =
19     false
20     ): MovieDto
21
22     @GET("movie/{movie_id}")
23     suspend fun fetchMovie(
24         @Path("movie_id") movieId: Int,
25         @Query("api_key") apiKey: String =
26     BuildConfig.apiKey
27     ): MovieDetailDto
28 }

```

Tabel 14. Jawaban Soal 1 MovieDto.kt

```

1 package com.example.retrofit.data.remote.models
2
3
4 import kotlinx.serialization.SerialName
5 import kotlinx.serialization.Serializable
6
7 @Serializable
8 data class MovieDto(
9     @SerialName("page")
10     val page: Int? = null,
11     @SerialName("results")
12     val results:
13 List<com.example.retrofit.data.remote.models.Result?>?
14 = null,
15     @SerialName("total_pages")
16     val totalPages: Int? = null,
17     @SerialName("total_results")
18     val totalResults: Int? = null
19 )

```

Tabel 15. Jawaban Soal 1 MovieDetailDto.kt

```

1 package com.example.retrofit.data.remote.models
2
3 import kotlinx.serialization.SerialName
4 import kotlinx.serialization.Serializable
5
6 @Serializable
7 data class MovieDetailDto(

```

```

8      @SerializedName("adult")
9      val adult: Boolean? = null,
10     @SerializedName("backdrop_path")
11     val backdropPath: String? = null,
12     @SerializedName("belongs_to_collection")
13     val belongsToCollection: BelongsToCollection? =
14     null,
15     @SerializedName("homepage")
16     val homepage: String? = null,
17     @SerializedName("id")
18     val id: Int? = null,
19     @SerializedName("genres")
20     val genres: List<Genre?>? = null,
21     @SerializedName("imdb_id")
22     val imdbId: String? = null,
23     @SerializedName("origin_country")
24     val originCountry: List<String?>? = null,
25     @SerializedName("original_language")
26     val originalLanguage: String? = null,
27     @SerializedName("original_title")
28     val originalTitle: String? = null,
29     @SerializedName("overview")
30     val overview: String? = null,
31     @SerializedName("popularity")
32     val popularity: Double? = null,
33     @SerializedName("poster_path")
34     val posterPath: String? = null,
35     @SerializedName("release_date")
36     val releaseDate: String? = null,
37     @SerializedName("revenue")
38     val revenue: Int? = null,
39     @SerializedName("runtime")
40     val runtime: Int? = null,
41     @SerializedName("status")
42     val status: String? = null,
43     @SerializedName("tagline")
44     val tagline: String? = null,
45     @SerializedName("title")
46     val title: String? = null,
47     @SerializedName("video")
48     val video: Boolean? = null,
49     @SerializedName("vote_average")
50     val voteAverage: Double? = null,
51     @SerializedName("vote_count")
52     val voteCount: Int? = null,
53 )

```

Tabel 16. Jawaban Soal 1 MovieRepositoryImpl.kt

```

1 package com.example.retrofit.data.repository_impl
2
3 import com.example.retrofit.common.data.ApiMapper
4 import
5 com.example.retrofit.common.data.MovieDetailMapper
6 import com.example.retrofit.data.local.dao.MovieDao
7 import com.example.retrofit.data.local.mapper.toDomain
8 import com.example.retrofit.data.local.mapper.toEntity
9 import
10 com.example.retrofit.data.remote.api.MovieApiService
11 import com.example.retrofit.data.remote.models.MovieDto
12 import com.example.retrofit.domain.models.Movie
13 import
14 com.example.retrofit.domain.repository.MovieRepository
15 import com.example.retrofit.utils.Response
16 import kotlinx.coroutines.flow.Flow
17 import kotlinx.coroutines.flow.flow
18 import com.example.retrofit.domain.models.MovieDetail
19 import kotlinx.coroutines.Dispatchers
20 import kotlinx.coroutines.withContext
21
22 class MovieRepositoryImpl(
23     private val movieApiService: MovieApiService,
24     private val apiMapper: ApiMapper<List<Movie>,
25 MovieDto>,
26     private val movieDetailMapper: MovieDetailMapper,
27     private val movieDao: MovieDao
28 ) : MovieRepository {
29     override fun fetchDiscoverMovie():
30 Flow<Response<List<Movie>>> = flow {
31     emit(Response.Loading())
32
33     val cachedMovies = movieDao.getAllMovies().map
34 { it.toDomain() }
35     if (cachedMovies.isNotEmpty()) {
36         emit(Response.Success(cachedMovies))
37     }
38
39     try {
40         val movieDto =
41 movieApiService.fetchDiscoverMovie()
42         val movies =
43 apiMapper.mapToDomain(movieDto)
44
45         withContext(Dispatchers.IO) {
46             movieDao.clearAll()

```

```

47         movieDao.insertMovies (movies.map {
48     it.toEntity() })
49     }
50
51     emit (Response.Success (movies))
52
53     } catch (e: Exception) {
54         if (cachedMovies.isEmpty()) {
55             emit (Response.Error (e))
56         }
57     }
58 }
59
60 override fun fetchMovie (movieId: Int):
61 Flow<Response<MovieDetail>> = flow {
62     emit (Response.Loading())
63     try {
64         val dto =
65 movieApiService.fetchMovie (movieId)
66         val movie =
67 movieDetailMapper.mapMovieDetailDtoToDomain (dto)
68         emit (Response.Success (movie))
69     } catch (e: Exception) {
70         emit (Response.Error (e))
71     }
72 }
73 }

```

Tabel 17. Jawaban Soal 1 DatabaseModule.kt

```

1 package com.example.retrofit.di
2
3 import android.content.Context
4 import androidx.room.Room
5 import com.example.retrofit.data.local.dao.MovieDao
6 import com.example.retrofit.data.local.db.MovieDatabase
7 import dagger.Module
8 import dagger.Provides
9 import dagger.hilt.InstallIn
10 import
11 dagger.hilt.android.qualifiers.ApplicationContext
12 import dagger.hilt.components.SingletonComponent
13 import javax.inject.Singleton
14
15 @Module
16 @InstallIn (SingletonComponent::class)
17 object DatabaseModule {

```

```

18
19     @Provides
20     @Singleton
21     fun provideAppDatabase(@ApplicationContext context:
22 Context): MovieDatabase {
23         return Room.databaseBuilder(
24             context,
25             MovieDatabase::class.java,
26             "movie_database"
27         ).fallbackToDestructiveMigration().build()
28     }
29
30     @Provides
31     @Singleton
32     fun provideMovieDao(appDatabase: MovieDatabase):
33 MovieDao {
34         return appDatabase.movieDao()
35     }
36 }

```

Tabel 18. Jawaban Soal 1 MovieModule.kt

```

1 package com.example.retrofit.di
2
3 import com.example.retrofit.common.data.ApiMapper
4 import
5 com.example.retrofit.common.data.MovieDetailMapper
6 import com.example.retrofit.data.local.dao.MovieDao
7 import
8 com.example.retrofit.data.mapper_impl.MovieApiMapperImp
9 l
10 import
11 com.example.retrofit.data.mapper_impl.MovieDetailMapper
12 Impl
13 import
14 com.example.retrofit.data.remote.api.MovieApiService
15 import com.example.retrofit.data.remote.models.MovieDto
16 import
17 com.example.retrofit.data.repository_impl.MovieRepositoryImpl
18 ryImpl
19 import com.example.retrofit.domain.models.Movie
20 import
21 com.example.retrofit.domain.repository.MovieRepository
22 import com.example.retrofit.utils.K
23 import
24 com.jakewharton.retrofit2.converter.kotlinx.serialization
25 on.asConverterFactory

```

```

26 import dagger.Module
27 import dagger.Provides
28 import dagger.hilt.InstallIn
29 import dagger.hilt.components.SingletonComponent
30 import kotlinx.serialization.json.Json
31 import okhttp3.MediaType.Companion.toMediaType
32 import retrofit2.Retrofit
33 import javax.inject.Singleton
34
35 @Module
36 @InstallIn(SingletonComponent::class)
37 object MovieModule {
38
39     private val json = Json{
40         coerceInputValues = true
41         ignoreUnknownKeys = true
42     }
43     @Provides
44     @Singleton
45     fun provideMovieRepository(
46         movieApiService: MovieApiService,
47         mapper: ApiMapper<List<Movie>, MovieDto>,
48         movieDetailMapper: MovieDetailMapper,
49         movieDao: MovieDao
50     ): MovieRepository = MovieRepositoryImpl(
51         movieApiService, mapper, movieDetailMapper,
52         movieDao
53     )
54
55
56     @Provides
57     @Singleton
58     fun provideMovieMapper(): ApiMapper<List<Movie>,
59     MovieDto> = MovieApiMapperImpl()
60
61     @Provides
62     @Singleton
63     fun provideMovieApiService(): MovieApiService{
64         val contentType =
65         "application/json".toMediaType()
66         return Retrofit.Builder()
67             .baseUrl(K.BASE_URL)
68
69         .addConverterFactory(json.asConverterFactory(contentType))
70         .build()
71         .create(MovieApiService::class.java)

```

```

72     }
73
74     @Provides
75     @Singleton
76     fun provideMovieDetailMapper(): MovieDetailMapper =
77     MovieDetailMapperImpl()
78 }

```

Tabel 19. Jawaban Soal 1 Movie.kt

```

1 package com.example.retrofit.domain.models
2
3 data class Movie(
4     val backdropPath: String,
5     val genreIds: String,
6     val id: Int,
7     val originalLanguage: String,
8     val originalTitle: String,
9     val overview: String,
10    val popularity: Double,
11    val posterPath: String,
12    val releaseDate: String,
13    val title: String,
14    val voteAverage: Double,
15    val voteCount: Int,
16    val video: Boolean
17 )

```

Tabel 20. Jawaban Soal 1 MovieDetail.kt

```

1 package com.example.retrofit.domain.models
2
3 data class MovieDetail(
4     val id: Int,
5     val title: String,
6     val overview: String,
7     val posterPath: String,
8     val releaseDate: String,
9     val genres: List<String>,
10    val voteAverage: Double,
11    val voteCount: Int,
12    val popularity: Double,
13    val backdropPath: String,
14    val originalLanguage: String,
15    val originalTitle: String,
16    val video: Boolean
17 )

```

Tabel 21. Jawaban Soal 1 MovieRepository.kt

1	package com.example.retrofit.domain.repository
2	
3	import com.example.retrofit.domain.models.Movie
4	import com.example.retrofit.domain.models.MovieDetail
5	import com.example.retrofit.utils.Response
6	import kotlinx.coroutines.flow.Flow
7	
8	interface MovieRepository {
9	fun fetchDiscoverMovie():
10	Flow<Response<List<Movie>>>
11	fun fetchMovie(movieId: Int):
12	Flow<Response<MovieDetail>>
13	}

Tabel 22. Jawaban Soal 1 DetailScreenViewModel.kt

1	package com.example.retrofit.presentation.detail
2	
3	import androidx.lifecycle.SavedStateHandle
4	import androidx.lifecycle.ViewModel
5	import androidx.lifecycle.viewModelScope
6	import com.example.retrofit.domain.models.MovieDetail
7	import
8	com.example.retrofit.domain.repository.MovieRepository
9	import com.example.retrofit.utils.Response
10	import dagger.hilt.android.lifecycle.HiltViewModel
11	import kotlinx.coroutines.flow.MutableStateFlow
12	import kotlinx.coroutines.flow.StateFlow
13	import kotlinx.coroutines.flow.asStateFlow
14	import kotlinx.coroutines.launch
15	import javax.inject.Inject
16	
17	@HiltViewModel
18	class DetailScreenViewModel @Inject constructor(
19	private val repository: MovieRepository,
20	savedStateHandle: SavedStateHandle
21	) : ViewModel() {
22	
23	private val movieId: Int =
24	savedStateHandle.get<String>("movieId")?.toIntOrNull()
25	?: 0
26	
27	private val _movie =
28	MutableStateFlow<Response<MovieDetail>>(Response.Loading())
29	g())
30	val movie: StateFlow<Response<MovieDetail>> =

```

31  _movie.asStateFlow()
32
33      init {
34          getMovieDetail()
35      }
36
37      private fun getMovieDetail() {
38          viewModelScope.launch {
39              repository.fetchMovie(movieId).collect {
40                  _movie.value = it
41              }
42          }
43      }
44  }

```

Tabel 23. Jawaban Soal 1 HomeViewModel.kt

```

1  package com.example.retrofit.presentation.home
2
3  import androidx.lifecycle.ViewModel
4  import androidx.lifecycle.viewModelScope
5  import com.example.retrofit.domain.models.Movie
6  import
7  com.example.retrofit.domain.repository.MovieRepository
8  import com.example.retrofit.utils.collectAndHandle
9  import dagger.hilt.android.lifecycle.HiltViewModel
10 import kotlinx.coroutines.flow.MutableStateFlow
11 import kotlinx.coroutines.flow.asStateFlow
12 import kotlinx.coroutines.flow.update
13 import kotlinx.coroutines.launch
14 import javax.inject.Inject
15
16 @HiltViewModel
17 class HomeViewModel @Inject constructor(
18     private val repository: MovieRepository,
19 ): ViewModel() {
20     private val _homeState =
21     MutableStateFlow(HomeState())
22     val homeState = _homeState.asStateFlow()
23
24     init {
25         fetchDiscoverMovie()
26     }
27
28     private fun fetchDiscoverMovie() =
29     viewModelScope.launch {
30

```

```

31 repository.fetchDiscoverMovie().collectAndHandle(
32     onError = { error ->
33         _homeState.update {
34             it.copy(
35                 error = error.message,
36                 isLoading = false
37             )
38         }
39     },
40     onLoading = {
41         _homeState.update {
42             it.copy(
43                 isLoading = true,
44                 error = null
45             )
46         }
47     },
48     ) { movies ->
49         _homeState.update {
50             it.copy(
51                 discoverMovies = movies,
52                 isLoading = false,
53                 error = null
54             )
55         }
56     }
57 }
58
59 fun retry() {
60     fetchDiscoverMovie()
61 }
62 }
63
64 data class HomeState(
65     val discoverMovies: List<Movie> = emptyList(),
66     val error: String? = null,
67     val isLoading: Boolean = false
68 )
69

```

Tabel 24. Jawaban Soal 1 HomeScreen.kt

```

1 package com.example.retrofit.presentation.home
2
3 import android.content.Intent
4 import android.net.Uri
5 import androidx.compose.foundation.layout.Arrangement

```

```

6   import androidx.compose.foundation.layout.Box
7   import androidx.compose.foundation.layout.Column
8   import androidx.compose.foundation.layout.Row
9   import androidx.compose.foundation.layout.Spacer
10  import androidx.compose.foundation.layout.fillMaxSize
11  import androidx.compose.foundation.layout.fillMaxWidth
12  import androidx.compose.foundation.layout.height
13  import androidx.compose.foundation.layout.padding
14  import androidx.compose.foundation.layout.width
15  import androidx.compose.foundation.lazy.LazyColumn
16  import androidx.compose.foundation.lazy.items
17  import
18  androidx.compose.foundation.shape.RoundedCornerShape
19  import androidx.compose.material3.Button
20  import androidx.compose.material3.Card
21  import
22  androidx.compose.material3.CircularProgressIndicator
23  import androidx.compose.material3.MaterialTheme
24  import androidx.compose.material3.Text
25  import androidx.compose.runtime.Composable
26  import androidx.compose.runtime.collectAsState
27  import androidx.compose.runtime.getValue
28  import androidx.compose.ui.Alignment
29  import androidx.compose.ui.Modifier
30  import androidx.compose.ui.draw.clip
31  import androidx.compose.ui.layout.ContentScale
32  import androidx.compose.ui.platform.LocalContext
33  import androidx.compose.ui.res.painterResource
34  import androidx.compose.ui.text.style.TextOverflow
35  import androidx.compose.ui.unit.dp
36  import androidx.hilt.navigation.compose.hiltViewModel
37  import androidx.navigation.NavController
38  import coil.compose.AsyncImage
39  import com.example.retrofit.R
40  import com.example.retrofit.domain.models.Movie
41  import com.example.retrofit.utils.K
42
43  @Composable
44  fun HomeScreen(
45      navController: NavController,
46      viewModel: HomeViewModel = hiltViewModel()
47  ) {
48      val context = LocalContext.current
49      val homeState by
50      viewModel.homeState.collectAsState()
51
52      Column(

```

```

53         modifier = Modifier
54             .fillMaxSize()
55             .padding(16.dp)
56     ) {
57         when {
58             homeState.isLoading -> {
59                 Box(
60                     modifier = Modifier.fillMaxSize(),
61                     contentAlignment =
62 Alignment.Center
63                 ) {
64                     CircularProgressIndicator()
65                 }
66             }
67
68             homeState.error != null -> {
69                 Column(
70                     modifier = Modifier.fillMaxSize(),
71                     verticalArrangement =
72 Arrangement.Center,
73                     horizontalAlignment =
74 Alignment.CenterHorizontally
75                 ) {
76                     Text(
77                         text = "Error:
78 ${homeState.error}",
79                         color =
80 MaterialTheme.colorScheme.error
81                     )
82                     Spacer(modifier =
83 Modifier.height(16.dp))
84                     Button(onClick = {
85                         viewModel.retry()
86                     }) {
87                         Text("Retry")
88                     }
89                 }
90             }
91
92             else -> {
93                 LazyColumn {
94                     items(homeState.discoverMovies) {
95 movie ->
96                         MovieCard(
97                             movie = movie,
98                             onDetailClick = {
99

```

```

100 navController.navigate("detail_screen/${movie.id}")
101     },
102     onWebsiteClick = {
103         val intent = Intent(
104
105             Intent.ACTION_VIEW,
106
107             Uri.parse("https://www.themoviedb.org/movie/${movie.id}
108             ")
109         )
110
111     context.startActivity(intent)
112     }
113 )
114 }
115 }
116 }
117 }
118 }
119 }
120
121 @Composable
122 fun MovieCard(
123     movie: Movie,
124     onDetailClick: () -> Unit,
125     onWebsiteClick: () -> Unit
126 ) {
127     Card(
128         modifier = Modifier
129             .padding(8.dp)
130             .fillMaxWidth()
131     ) {
132         Row(modifier = Modifier.padding(16.dp)) {
133             AsyncImage(
134                 model =
135                 "${K.BASE_IMAGE_URL}${movie.posterPath}",
136                 contentDescription = movie.title,
137                 contentScale = ContentScale.Crop,
138                 modifier = Modifier
139                     .height(130.dp)
140                     .width(100.dp)
141                     .align(Alignment.CenterVertically)
142                     .clip(RoundedCornerShape(10.dp)),
143                 placeholder =
144                 painterResource(R.drawable.ic_launcher_foreground),
145                 error =
146                 painterResource(R.drawable.ic_launcher_foreground)

```

```

147         )
148
149         Column(
150             modifier = Modifier
151                 .padding(start = 16.dp)
152                 .fillMaxWidth()
153         ) {
154             Text(
155                 text = movie.title,
156                 style =
157                 MaterialTheme.typography.titleMedium
158             )
159
160             Text(
161                 text = movie.overview,
162                 style =
163                 MaterialTheme.typography.bodyMedium,
164                 maxLines = 3,
165                 overflow = TextOverflow.Ellipsis
166             )
167
168             Text(
169                 text = "Released:
170                 ${movie.releaseDate}",
171                 style =
172                 MaterialTheme.typography.bodySmall,
173                 modifier = Modifier.padding(top =
174                 4.dp)
175             )
176
177             Text(
178                 text = "Rating:
179                 ${String.format("%.1f", movie.voteAverage)}/10",
180                 style =
181                 MaterialTheme.typography.bodySmall,
182                 modifier = Modifier.padding(top =
183                 2.dp)
184             )
185
186             Row(
187                 modifier = Modifier
188                     .padding(top = 8.dp)
189                     .fillMaxWidth(),
190                 horizontalArrangement =
191                 Arrangement.SpaceBetween
192             ) {
193                 Button(onClick = onDetailClick) {

```

194	Text("Detail")
195	}
196	
197	Button(onClick = onWebsiteClick) {
198	Text("TMDB")
199	}
200	}
201	}
202	}
203	}
204	}

Tabel 25. Jawaban Soal 1 DetailScreen.kt

1	package com.example.retrofit.presentation.detail
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import androidx.compose.foundation.rememberScrollState
6	import androidx.compose.foundation.verticalScroll
7	import androidx.compose.material.icons.Icons
8	import
9	androidx.compose.material.icons.filled.ArrowBack
10	import androidx.compose.material3.*
11	import androidx.compose.runtime.*
12	import androidx.compose.ui.Alignment
13	import androidx.compose.ui.Modifier
14	import androidx.compose.ui.layout.ContentScale
15	import androidx.compose.ui.text.font.FontWeight
16	import androidx.compose.ui.text.style.TextAlign
17	import androidx.compose.ui.unit.dp
18	import androidx.compose.ui.unit.sp
19	import androidx.hilt.navigation.compose.hiltViewModel
20	import androidx.navigation.NavController
21	import coil.compose.rememberImagePainter
22	import com.example.retrofit.utils.K
23	import com.example.retrofit.utils.Response
24	
25	@Composable
26	fun DetailScreen(
27	movieId: Int,
28	navController: NavController,
29	onBackClick: () -> Unit,
30	viewModel: DetailScreenViewModel = hiltViewModel()
31	) {
32	val movieState by viewModel.movie.collectAsState()
33	

```

34     when (movieState) {
35         is Response.Loading -> {
36             Box(
37                 modifier = Modifier.fillMaxSize(),
38                 contentAlignment = Alignment.Center
39             ) {
40                 CircularProgressIndicator()
41             }
42         }
43
44         is Response.Success -> {
45             val movie = (movieState as
46 Response.Success).data
47
48             Box(modifier = Modifier.fillMaxSize()) {
49                 Column(
50                     modifier = Modifier
51                         .fillMaxSize()
52
53 .verticalScroll(rememberScrollState())
54                 ) {
55                     Box(
56                         modifier = Modifier
57                             .fillMaxWidth()
58                             .height(300.dp)
59                     ) {
60                         Image(
61                             painter =
62 rememberImagePainter("https://image.tmdb.org/t/p/w780$
63 {movie.backdropPath}"),
64                             contentDescription = null,
65                             contentScale =
66 ContentScale.Crop,
67                             modifier = Modifier
68                                 .fillMaxWidth()
69                                 .height(220.dp)
70                         )
71                         Card(
72                             modifier = Modifier
73                                 .padding(start =
74 16.dp)
75                                 .width(120.dp)
76                                 .height(180.dp)
77
78 .align(Alignment.BottomStart),
79                             shape =
80 MaterialTheme.shapes.medium,

```

```

81         elevation =
82 CardDefaults.cardElevation(8.dp)
83     ) {
84         Image(
85             painter =
86 rememberImagePainter("${K.BASE_IMAGE_URL}${movie.posterPath}"),
87             contentDescription =
88 "Poster",
89             contentScale =
90 ContentScale.Crop,
91             modifier =
92 Modifier.fillMaxSize()
93         )
94     }
95 }
96 }
97 Column(
98     modifier = Modifier
99         .fillMaxWidth()
100         .padding(horizontal =
101 16.dp)
102         .padding(top = 16.dp) //
103 Add top padding to avoid overlap
104 ) {
105     Row(
106         modifier =
107 Modifier.fillMaxWidth(),
108         horizontalArrangement =
109 Arrangement.spacedBy(16.dp)
110     ) {
111         Column(
112             modifier =
113 Modifier.weight(1f)
114         ) {
115             Text(
116                 text =
117 movie.title,
118                 fontSize = 20.sp,
119                 fontWeight =
120 FontWeight.Bold,
121                 maxLines = 2
122             )
123         }
124         Spacer(modifier =
125 Modifier.height(4.dp))
126     }
127     Text(

```



```

175                                                     fontSize =
176 12.sp
177                                     )
178                                     }
179                                 )
180                             }
181                         }
182
183                         Spacer(modifier =
184 Modifier.height(24.dp))
185
186                         Text(
187                             text = "Overview",
188                             fontWeight =
189 FontWeight.SemiBold,
190                             fontSize = 18.sp
191                         )
192
193                         Spacer(modifier =
194 Modifier.height(12.dp))
195
196                         Text(
197                             text = movie.overview,
198                             fontSize = 14.sp,
199                             textAlign =
200 TextAlign.Justify,
201                             lineHeight = 20.sp
202                         )
203
204                         Spacer(modifier =
205 Modifier.height(32.dp))
206                     }
207                 }
208
209                 Card(
210                     modifier = Modifier
211                         .padding(16.dp)
212                         .align(Alignment.TopStart),
213                     shape =
214 MaterialTheme.shapes.small,
215                     colors = CardDefaults.cardColors(
216                         containerColor =
217 MaterialTheme.colorScheme.surface.copy(alpha = 0.9f)
218                     )
219                 ) {
220                     IconButton(
221                         onClick = onBackClick,

```

```

222                                     modifier =
223 Modifier.size(40.dp)
224                                     ) {
225                                     Icon(
226                                     imageVector =
227 Icons.Default.ArrowBack,
228                                     contentDescription =
229 "Back",
230                                     tint =
231 MaterialTheme.colorScheme.onSurface
232                                     )
233                                     }
234                                     }
235                                     }
236                                     }
237
238         is Response.Error -> {
239             val error = (movieState as
240 Response.Error).error
241             Box(
242                 modifier = Modifier.fillMaxSize(),
243                 contentAlignment = Alignment.Center
244             ) {
245                 Column(
246                     horizontalAlignment =
247 Alignment.CenterHorizontally
248                 ) {
249                     Text(
250                         text = "Something went wrong",
251                         fontSize = 18.sp,
252                         fontWeight = FontWeight.Medium
253                     )
254                     Spacer(modifier =
255 Modifier.height(8.dp))
256                     Text(
257                         text = error?.message ?:
258 "Unknown Error",
259                         fontSize = 14.sp,
260                         color =
261 MaterialTheme.colorScheme.onSurface.copy(alpha =
262 0.6f),
263                         textAlign = TextAlign.Center
264                     )
265                 }
266             }
267         }

```

268	}
269	}

Tabel 26. Jawaban Soal 1 AppNavigation.kt

1	package com.example.retrofit.presentation.navigation
2	
3	import androidx.compose.foundation.layout.padding
4	import androidx.compose.material3.Scaffold
5	import androidx.compose.runtime.Composable
6	import androidx.compose.ui.Modifier
7	import androidx.navigation.compose.NavHost
8	import androidx.navigation.compose.composable
9	import
10	androidx.navigation.compose.rememberNavController
11	import
12	com.example.retrofit.presentation.detail.DetailScreen
13	import
14	com.example.retrofit.presentation.home.HomeScreen
15	
16	@Composable
17	fun AppNavHost() {
18	val navController = rememberNavController()
19	
20	Scaffold { innerPadding ->
21	NavHost(
22	navController = navController,
23	startDestination = "home_screen",
24	modifier = Modifier.padding(innerPadding)
25	) {
26	composable("home_screen") {
27	HomeScreen(navController =
28	navController)
29	}
30	
31	composable("detail_screen/{movieId}") {
32	backStackEntry ->
33	val movieId =
34	backStackEntry.arguments?.getString("movieId")?.toIntO
35	rNull() ?: 0
36	DetailScreen(
37	movieId = movieId,
38	navController = navController,
39	onBackClick = {
40	navController.popBackStack() }
41	)
42	}

43	}
44	}
45	}

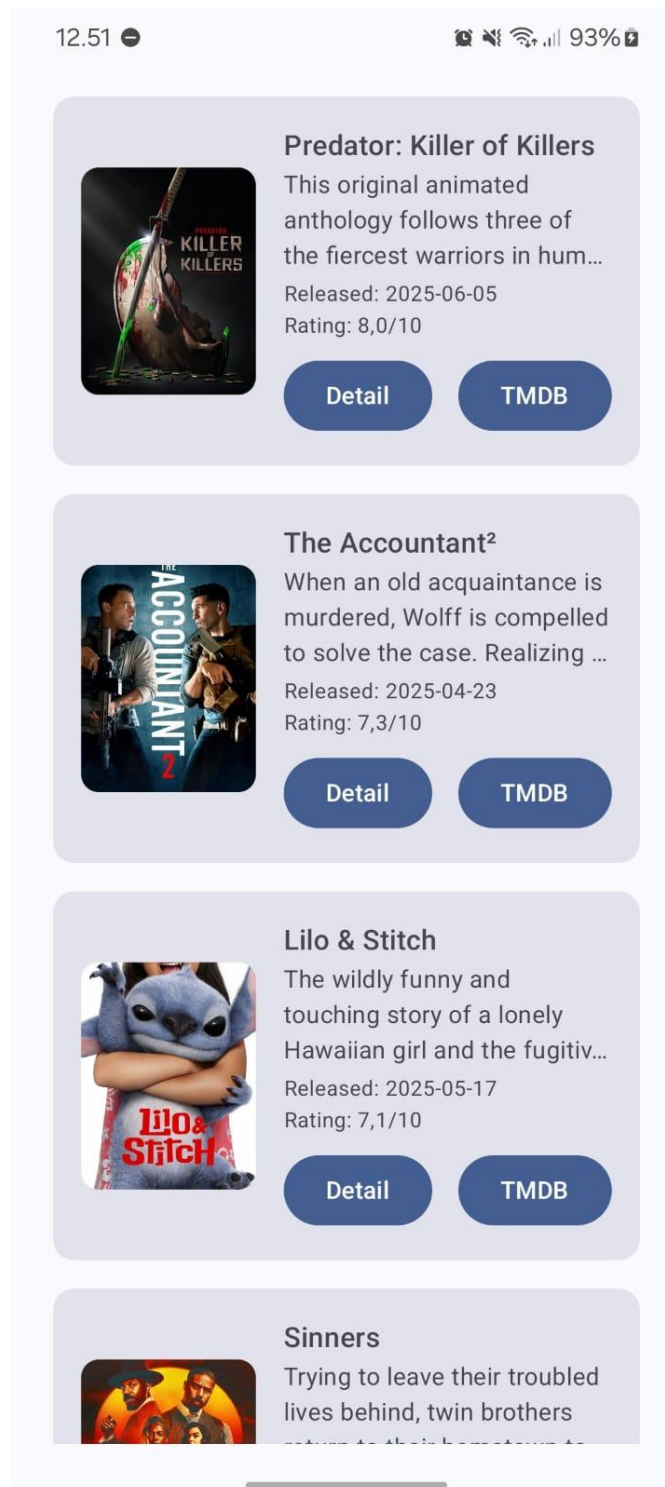
Tabel 27. Jawaban Soal 1 MovieApplication.kt

1	package com.example.retrofit
2	
3	import android.app.Application
4	import dagger.hilt.android.HiltAndroidApp
5	
6	@HiltAndroidApp
7	class MovieApplication: Application() {
8	
9	}

Tabel 28. Jawaban Soal 1 MainActivity.kt

1	package com.example.retrofit
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import
8	com.example.retrofit.presentation.navigation.AppNavHost
9	t
10	import
11	com.example.retrofit.presentation.theme.RetrofitTheme
12	import dagger.hilt.android.AndroidEntryPoint
13	
14	@AndroidEntryPoint
15	class MainActivity : ComponentActivity() {
16	override fun onCreate(savedInstanceState: Bundle?)
17	{
18	super.onCreate(savedInstanceState)
19	enableEdgeToEdge()
20	setContent {
21	RetrofitTheme {
22	AppNavHost()
23	}
24	}
25	}
26	}

B. Output Program



Gambar 1. Screenshot Home Screen Soal 1



Gambar 2. Screenshot Detail Screen Soal 1



Gambar 3. Screenshot Intent Browser Soal 1

C. Pembahasan

Ext.kt

Baris [1], menandakan file ini berada dalam package `com.example.retrofit.utils`. Baris [3–4] mengimpor library yang dibutuhkan, yaitu untuk logging (Log) dan aliran data menggunakan Kotlin Flow.

Baris [6–24] merupakan Fungsi `collectAndHandle` dan ekstensi dari `Flow<Response<T>>` yang bersifat suspend, artinya hanya bisa dipanggil di dalam coroutine. Fungsi ini digunakan untuk mengumpulkan data dari Flow dan menangani tiga kemungkinan status respon: Loading, Success, dan Error. Kemudian, di dalam blok collect, when digunakan untuk memeriksa tipe Response yang diterima, lalu mengeksekusi aksi sesuai status tersebut.

GenreConstant.kt

Baris [1] menyatakan bahwa file ini berada dalam package `com.example.retrofit.utils`. Kemudian, baris [3 – 22] mendefinisikan object `GenreConstants` yang berfungsi sebagai tempat penyimpanan data konstan berupa genre film. Di dalamnya terdapat `genreMap`, yaitu sebuah Map yang memetakan ID genre (bertipe Int) ke nama genre (bertipe String), misalnya 28 ke "Action" dan 35 ke "Comedy". Map ini bersifat immutable (tidak bisa diubah) dan hanya bisa diakses dalam object ini karena diberi modifier `private`.

Baris [26 – 28] berisi fungsi `getGenreNameById()` yang menerima ID genre sebagai parameter, dan mengembalikan nama genre berdasarkan `genreMap`. Jika ID tidak ditemukan, maka akan mengembalikan "Unknown" sebagai nilai default.

K.kt

Baris [1] menyatakan bahwa file ini berada dalam package `com.example.retrofit.utils`. Kemudian, baris [3 – 69] mendefinisikan object `K` yang berisi konstanta string yang digunakan dalam komunikasi dengan API The Movie Database (TMDB). `BASE_URL` adalah URL dasar untuk semua endpoint TMDB, `BASE_IMAGE_URL` digunakan untuk mengambil gambar poster film dengan ukuran lebar 500px, dan `MOVIE_ENDPOINT` menunjuk ke endpoint khusus untuk mendapatkan daftar film.

Response.kt

Baris [1] menyatakan bahwa file ini berada dalam package `com.example.retrofit.utils`. Baris [3 – 7] mendefinisikan kelas `Response` sebagai sealed class yang generic

(menggunakan tipe T). Kelas ini digunakan untuk merepresentasikan tiga kemungkinan status dari proses asynchronous seperti pemanggilan API. Pertama, mendefinisikan subclass Success yang menyimpan data hasil response, Kedua, mendefinisikan subclass Error yang menyimpan error (jika ada) dan data opsional. Ketiga, mendefinisikan subclass Loading untuk menunjukkan bahwa proses sedang berlangsung.

ApiMapper.kt

Baris [1] menyatakan bahwa file ini berada dalam package `com.example.retrofit.common.data`. Kemudian, baris [3 – 5] mendefinisikan interface `ApiMapper` yang bersifat generic dengan dua tipe: `Domain` dan `Entity`. Interface ini digunakan sebagai kontrak untuk memetakan data dari model API (`Entity`) ke model domain (`Domain`) melalui fungsi `mapToDomain()`.

MovieDetailMapper.kt

Baris [1], menandakan bahwa file ini berada dalam package `com.example.retrofit.common.data`. Pada baris [3 – 5], dilakukan import terhadap dua model data: `MovieDetailDto` dari lapisan data (data transfer object/DTO) dan `MovieDetail` dari lapisan domain. Kedua model ini digunakan untuk proses pemetaan data dari bentuk yang diterima dari API menjadi bentuk yang lebih sesuai untuk digunakan dalam aplikasi.

Interface `MovieDetailMapper` [baris 7 – 9] didefinisikan sebagai kontrak untuk pemetaan data detail film. Di dalamnya terdapat satu fungsi bernama `mapMovieDetailDtoToDomain()` yang bertugas mengubah objek `MovieDetailDto` (yang biasanya langsung didapat dari response API) menjadi objek `MovieDetail` yang merupakan representasi data di domain layer.

MovieDao

Baris [1] menyatakan lokasi file dalam package `com.example.retrofit.data.local.dao`. Baris [3–9] mengimpor komponen-komponen penting dari Room seperti `@Dao`, `@Query`, dan `@Insert`, serta entitas `MovieEntity` dan `Flow` dari Kotlin Coroutines yang digunakan untuk mendukung pemrosesan data secara reaktif.

Baris [11] mendefinisikan `MovieDao` sebagai interface DAO yang digunakan untuk mengakses dan memodifikasi data film dalam database lokal. Baris [14 – 16] berisi fungsi `getAllMovies()` yang mengambil seluruh data film secara suspend dan mengurutkannya berdasarkan popularitas. Baris [18 – 20] menyediakan versi reaktif dari fungsi tersebut

melalui Flow, sehingga memungkinkan UI menerima pembaruan data secara otomatis. Selanjutnya, baris [22 – 23] mendefinisikan fungsi `insertMovies()` yang menyimpan daftar film ke database dan akan menggantikan data lama jika primary key-nya sama. Baris [25 – 26] berisi fungsi `clearAll()` untuk menghapus seluruh data film, dan baris [28 – 29] menyediakan fungsi `getMoviesCount()` yang menghitung jumlah film yang tersimpan di database.

MovieDatabase.kt

Baris [1] menyatakan bahwa file ini berada dalam package `com.example.retrofit.data.local.db`. Baris [3–5] mengimpor anotasi dan kelas yang diperlukan dari Room untuk mendefinisikan database, serta `MovieDao` sebagai akses data.

Baris [7] menggunakan anotasi `@Database` untuk mendeklarasikan bahwa `MovieDatabase` adalah kelas database Room. Di sini, hanya terdapat satu entitas, yaitu `MovieEntity`, dan versi database diset ke 2. Baris [8 – 11] mendefinisikan kelas abstrak `MovieDatabase` yang merupakan turunan dari `RoomDatabase`, dan menyediakan fungsi abstrak `movieDao()` yang mengembalikan instance dari `MovieDao` untuk digunakan dalam operasi database.

MovieEntity.kt

Baris [1] menunjukkan bahwa file ini berada dalam package `com.example.retrofit.data.local.entities`. Baris [3–4] mengimpor anotasi dari Room yang diperlukan untuk mendefinisikan sebuah entitas tabel dan menentukan primary key-nya. Baris [6] mendeklarasikan `MovieEntity` sebagai data class dan menandainya dengan anotasi `@Entity`, yang berarti class ini akan direpresentasikan sebagai tabel bernama "movies" di database lokal. Baris [7–20] mendefinisikan properti-properti dari `MovieEntity`, yang mencerminkan informasi film seperti `id`, `title`, `overview`, `releaseDate`, `voteAverage`, hingga `genreIds`. Properti `@PrimaryKey` ditetapkan pada `id` untuk menandai kolom unik utama. Semua data ini akan disimpan sebagai satu baris dalam tabel `movies`.

MovieLocalMapper.kt

Baris [1] menunjukkan bahwa file ini berada dalam package `com.example.retrofit.data.local.mapper`. Baris [3 – 5] mengimpor model `MovieEntity` dari

layer lokal dan Movie dari domain, yang akan digunakan dalam proses konversi data antar lapisan.

Baris [7 – 23] mendefinisikan ekstensi fungsi `toDomain()` pada `MovieEntity`, yang bertugas mengubah objek `MovieEntity` menjadi objek `Movie`. Ini memungkinkan data dari database lokal dapat digunakan di layer domain atau UI dengan struktur yang sesuai. Sebaliknya, baris [25 – 40] mendefinisikan fungsi `toEntity()` pada objek `Movie`, yang mengonversi data dari domain menjadi `MovieEntity` agar bisa disimpan kembali ke database lokal.

MovieApiMapperImpl.kt

Baris [1] menunjukkan bahwa file ini berada dalam package `com.example.retrofit.data.mapper_impl`. Baris [3–6] mengimpor interface `ApiMapper`, `DTO` `MovieDto`, model domain `Movie`, serta `GenreConstants` yang digunakan untuk memetakan ID genre ke nama genre.

Baris [8] mendefinisikan kelas `MovieApiMapperImpl` yang mengimplementasikan `ApiMapper` untuk mengonversi `MovieDto` menjadi list `Movie`. Baris [10 – 42] mengimplementasikan fungsi `mapToDomain()`, yang memetakan setiap elemen dalam `results` dari `MovieDto` ke dalam objek `Movie`. Dalam proses ini, nilai null atau kosong akan diformat menggunakan fungsi `formatEmptyValue()` untuk mencegah crash dan memberikan nilai default seperti "Unknown title" atau "Unknown date". Sementara itu, ID genre diubah menjadi string nama genre yang dipisahkan koma menggunakan fungsi `formatGenre()` yang memanfaatkan `GenreConstants`.

Baris [44 – 58] mendefinisikan fungsi `formatEmptyValue()` yang memeriksa apakah nilai string kosong atau null, dan mengembalikan string default jika iya. Baris [33–36] mendefinisikan fungsi `formatGenre()`, yang akan mengubah list ID genre menjadi nama-nama genre dalam satu string, atau mengembalikan "Unknown Genre" jika datanya null.

MovieDetailMapperImpl.kt

Baris [1] menunjukkan bahwa file ini berada dalam package `com.example.retrofit.data.mapper_impl`. Baris [3 – 7] mengimpor interface

MovieDetailMapper, DTO MovieDetailDto dari layer data remote, serta model MovieDetail dari layer domain.

Baris [9] mendefinisikan kelas MovieDetailMapperImpl yang mengimplementasikan MovieDetailMapper, yang bertanggung jawab mengonversi MovieDetailDto menjadi objek MovieDetail. Baris [10 – 30] merupakan implementasi fungsi mapMovieDetailDtoToDomain(), di mana setiap properti dari dto dicek nilainya, jika null, maka akan diganti dengan nilai default seperti -1 untuk ide, string kosong untuk teks, 0.0 untuk angka, atau false untuk boolean. Properti genres dipetakan menjadi list string nama genre, dengan fallback “Unknown” jika ada genre yang null.

MovieApiService.kt

Baris [1] menyatakan bahwa file ini berada dalam package com.example.retrofit.data.remote.api. Baris [3 – 10] mengimpor objek BuildConfig untuk mendapatkan API key, dua model DTO (MovieDetailDto dan MovieDto) untuk menerima respons dari server, konstanta dari K sebagai endpoint, serta anotasi Retrofit seperti @GET, @Path, dan @Query.

Baris [12] mendefinisikan interface MovieApiService sebagai deklarasi endpoint API yang akan digunakan Retrofit. Baris [14 – 20] adalah fungsi fetchDiscoverMovie() untuk mengambil daftar film dari endpoint discover/movie. Fungsi ini menggunakan API key dari BuildConfig dan menyertakan parameter opsional include_adult yang diset default false. Responsnya akan dipetakan ke dalam MovieDto. Selanjutnya, baris [22 – 28] mendefinisikan fungsi fetchMovie() yang mengambil detail dari film tertentu berdasarkan movie_id. ID tersebut disisipkan ke URL melalui anotasi @Path, dan API key tetap dikirim melalui @Query. Responsnya akan dikembalikan sebagai objek MovieDetailDto.

MovieDto.kt

Baris [1] menunjukkan bahwa file ini berada dalam package com.example.retrofit.data.remote.models. Baris [4–5] mengimpor anotasi @Serializable dan @SerializedName dari Kotlin Serialization, yang memungkinkan objek ini untuk diserialisasi atau dideserialisasi dari/ke format JSON.

Baris [7] mendeklarasikan MovieDto sebagai data class yang dapat diserialisasi. Baris [8 – 18] mendefinisikan properti-properti dari respons API yang berkaitan dengan daftar film, yaitu page, results, totalPages, dan totalResults. Masing-masing properti diberi

anotasi `@SerializedName` untuk mencocokkan nama field JSON dari server. Properti `results` merupakan list dari objek `Result`, yang berisi data film individual. Semua properti diberi nilai default null untuk menangani kemungkinan nilai kosong dari respons API.

MovieDetailDto.kt

Baris [1] menyatakan bahwa file ini berada dalam package `com.example.retrofit.data.remote.models`. Baris [3–4] mengimpor anotasi `@Serializable` dan `@SerializedName` dari Kotlin Serialization, yang memungkinkan data class ini dikonversi dari/ke format JSON.

Baris [6] mendeklarasikan `MovieDetailDto` sebagai data class yang dapat diserialisasi dan mewakili detail lengkap dari sebuah film berdasarkan respons API. Baris [7 – 53] mendefinisikan properti-properti dari film seperti `adult`, `backdropPath`, `genres`, `homepage`, `id`, `overview`, `posterPath`, `releaseDate`, `runtime`, `status`, `title`, hingga `rating` dan jumlah suara (`voteAverage` dan `voteCount`). Setiap properti diberi anotasi `@SerializedName` untuk mencocokkan nama field dalam JSON dengan nama variabel Kotlin. Semua properti dibuat nullable dan diberi default null untuk memastikan proses deserialisasi tetap berjalan meskipun ada field yang hilang dalam respons API.

MovieRepositoryImpl.kt

Baris [1 – 20] berisi deklarasi package dan impor berbagai dependensi penting, seperti interface `ApiMapper`, `MovieDetailMapper`, akses DAO (`MovieDao`), fungsi konversi entitas ke domain (`toDomain`, `toEntity`), API service, model DTO (`MovieDto`), model domain (`Movie`, `MovieDetail`), interface repository (`MovieRepository`), kelas `Response`, serta coroutine utilities (`Flow`, `flow`, `Dispatchers`, `withContext`).

Baris [15 – 28] mendefinisikan class `MovieRepositoryImpl` yang mengimplementasikan `MovieRepository`, dan menerima empat parameter: `movieApiService` untuk komunikasi jaringan, `apiMapper` untuk konversi data API ke domain, `movieDetailMapper` untuk memetakan detail film, serta `movieDao` untuk mengakses database lokal.

Baris [29 – 58] mendefinisikan fungsi `fetchDiscoverMovie()` yang mengembalikan aliran `Flow` bertipe `Response<List<Movie>>`. Fungsi mulai dengan mengirim `Response.Loading()`. Kemudian, data film diambil dari database lokal dan jika tidak kosong, langsung dikirim sebagai `Response.Success`. Kemudian, mencoba mengambil data dari API,

mengonversinya dengan apiMapper, lalu menyimpannya ke database secara async di dalam blok `withContext(Dispatchers.IO)`. Data yang baru diambil kemudian dikirim sebagai `Response.Success`. Jika terjadi exception dan data lokal kosong, maka akan dikirim `Response.Error`.

Baris [60 – 72] adalah fungsi `fetchMovie(movieId: Int)` yang juga menggunakan flow. Ini mengirim `Response.Loading()`, lalu mengambil detail film dari API, memetakannya ke domain model, dan mengirimkannya sebagai `Response.Success`. Jika ada error, maka akan dikirim `Response.Error`.

DatabaseModule.kt

Baris [3 – 6] mengimpor `Context`, `Room`, `MovieDao`, dan `MovieDatabase` yang dibutuhkan untuk pembuatan dan akses database lokal. Kemudian, baris [7 – 13] mengimpor anotasi dan komponen dari Dagger Hilt seperti `@Module`, `@Provides`, `@InstallIn`, dan `@Singleton`, serta qualifier `@ApplicationContext`.

Baris [15 – 17] mendefinisikan objek `DatabaseModule` sebagai modul Hilt yang dipasang di komponen `SingletonComponent`, artinya dependensi akan bertahan selama aplikasi berjalan. Baris [19 – 28] mendefinisikan fungsi `provideAppDatabase()` yang membuat instance `Room` database dengan nama "movie_database", dan menggunakan `fallbackToDestructiveMigration()` untuk reset otomatis jika versi berubah. Diberi anotasi `@Provides` dan `@Singleton` agar dapat diinject dan hanya dibuat sekali. Baris [30 – 36] mendefinisikan fungsi `provideMovieDao()` yang mengembalikan DAO dari instance database yang dibuat. Juga diberi anotasi `@Provides` dan `@Singleton`.

MovieModule.kt

Baris [12 – 33] mengimpor semua dependensi yang diperlukan seperti interface, model, implementasi mapper dan repository, serta utilitas Retrofit dan Hilt. Kemudian juga mendefinisikan `@Module` Hilt `MovieModule` yang akan digunakan selama siklus hidup aplikasi.

Baris [35 – 42] membuat konfigurasi `Json` untuk konversi data menggunakan `kotlinx.serialization`, dengan dua opsi: mengabaikan field yang tidak dikenal dan memaksa nilai default jika tidak cocok. Baris [43 – 53] fungsi `provideMovieRepository()` menyediakan objek `MovieRepository` menggunakan implementasi `MovieRepositoryImpl`, dengan dependensi `MovieApiService`, `ApiMapper`, `MovieDetailMapper`, dan `MovieDao`.

Baris [56 – 59] fungsi `provideMovieMapper()` menyediakan instance `MovieApiMapperImpl` sebagai implementasi dari `ApiMapper`. Baris [61 – 72] fungsi `provideMovieApiService()` membangun dan menyediakan Retrofit instance dengan base URL dari konstanta `K.BASE_URL`, converter menggunakan konfigurasi JSON sebelumnya, dan menghasilkan `MovieApiService`.

Baris [74 – 78] fungsi `provideMovieDetailMapper()` menyediakan instance `MovieDetailMapperImpl` sebagai implementasi dari `MovieDetailMapper`.

Movie.kt

Baris [1] menyatakan bahwa file ini berada di package `com.example.retrofit.domain.models`. Baris [3–16] mendefinisikan data class `Movie` yang merepresentasikan model film di domain layer, berisi properti seperti, `id`, `title`, `overview`, `releaseDate`, `posterPath`, `backdropPath`, `originalTitle`, `originalLanguage`, `genreIds`, `popularity`, `voteAverage`, `voteCount` — metrik penilaian, dan `video`.

MovieDetail.kt

Baris [1] menunjukkan bahwa file ini berada di package `com.example.retrofit.domain.models`. Kemudian, baris [3–16] mendefinisikan data class `MovieDetail`, yang mewakili data detail sebuah film untuk ditampilkan di domain/UI. Properti yang dimuat meliputi, `id`, `title`, `overview`, `posterPath`, `releaseDate`, `genres`, `voteAverage`, `voteCount`, `popularity`, `backdropPath`, `originalLanguage`, `originalTitle`, `video`.

MovieRepository.kt

Baris [1] menyatakan lokasi file dalam package `com.example.retrofit.domain.repository`. Baris [3 – 6] mengimpor model `Movie`, `MovieDetail`, wrapper response `Response`, dan `Flow` dari Kotlin untuk pemrosesan data reaktif. Baris [8] mendeklarasikan interface `MovieRepository` yang menjadi kontrak atau abstraksi untuk data film yang bisa diakses oleh domain layer.

Baris [9 - 10] fungsi `fetchDiscoverMovie()` mengembalikan daftar film populer dalam bentuk `Flow<Response<List<Movie>>>`. Baris [11 - 12] fungsi `fetchMovie(movieId: Int)` mengembalikan detail dari film tertentu berdasarkan ID, juga dalam bentuk `Flow<Response<MovieDetail>>`.

DetailScreenViewModel.kt

Baris [1 – 9] mengimpor class dari Android Lifecycle (SavedStateHandle, ViewModel, viewModelScope) dan domain (MovieDetail, MovieRepository), serta Response sebagai wrapper state. Baris [10 – 15] mengimpor dependensi dari Hilt (@HiltViewModel, @Inject) dan Kotlin Flow (MutableStateFlow, StateFlow, asStateFlow, launch).

Baris [17 - 21] mendeklarasikan class DetailScreenViewModel yang dianotasi @HiltViewModel untuk dependency injection. Kemudian membuat konstruktor dengan injection MovieRepository dan SavedStateHandle untuk mengambil movieId dari argumen navigasi. Baris [23 - 25] mengambil movieId dari SavedStateHandle dengan fallback ke 0 jika null atau tidak valid. Baris [27 - 29] membuat _movie sebagai MutableStateFlow dengan nilai awal Response.Loading() — menandakan data sedang dimuat. Baris [30 - 21] mengekspos _movie sebagai StateFlow agar hanya bisa dibaca dari luar (immutable dari luar ViewModel).

Baris [33 - 35] saat ViewModel diinisialisasi, langsung memanggil getMovieDetail() untuk mengambil data detail film. Baris [37 - 42] fungsi getMovieDetail() menjalankan coroutine di viewModelScope dan mengoleksi aliran fetchMovie(movieId) dari repository, lalu memperbarui _movie dengan respons dari repository (baik sukses atau error).

HomeViewModel.kt

Baris [1 – 14] berisi deklarasi package dan impor berbagai dependensi penting seperti interface ApiMapper, MovieDetailMapper, DAO (MovieDao), fungsi konversi entitas (toDomain, toEntity), service API, model DTO dan domain, serta kelas pendukung seperti Response, Flow, dan coroutine utilities. Baris [15 – 26] mendefinisikan class MovieRepositoryImpl yang mengimplementasikan MovieRepository dan menerima empat dependency utama melalui constructor.

Baris [28 – 58] mendefinisikan fungsi fetchDiscoverMovie() yang mengirim Response.Loading(), membaca data dari database lokal, lalu mencoba mengambil data dari API, menyimpannya secara async ke database, dan mengembalikan data baru sebagai Response.Success, serta menangani error jika cache kosong. Baris [64 – 68] mendefinisikan data class HomeState, yang merepresentasikan state UI layar Home. Terdiri dari tiga properti: discoverMovies sebagai daftar film, error sebagai pesan kesalahan (jika ada), dan isLoading

sebagai penanda proses loading. Struktur state ini mendukung pendekatan UI yang reaktif dan deklaratif.

HomeScreen.kt

Baris [1 – 41] merupakan bagian impor yang menyertakan berbagai komponen dari Jetpack Compose, seperti Column, LazyColumn, Button, dan Text, serta komponen dari Material3. Selain itu, juga diimpor coil.compose.AsyncImage untuk memuat gambar secara asinkron, Intent dan Uri untuk membuka URL eksternal, dan komponen dari Hilt untuk dependency injection. Impor ini mendukung fungsi utama tampilan dan pengelolaan data pada layar Home. Baris [43 – 47] mendefinisikan fungsi HomeScreen() yang menerima NavController untuk navigasi dan HomeViewModel yang di-inject otomatis dengan Hilt. Baris [48 – 50] mengakses Context untuk membuka website eksternal, dan memantau state homeState dari ViewModel dengan collectAsState(). Baris [52 – 56] membuat layout utama dalam bentuk Column, dengan padding dan ukuran layar penuh. Baris [57 – 66] menangani kondisi saat loading: menampilkan CircularProgressIndicator yang diposisikan di tengah layar menggunakan Box. Baris [68 – 90] menangani kondisi error: menampilkan pesan kesalahan dan tombol "Retry" untuk memicu pemanggilan ulang API melalui viewModel.retry().

Baris [92 - 115] menangani kondisi sukses (tidak loading dan tidak error): menampilkan daftar film (discoverMovies) dalam LazyColumn, masing-masing di-render melalui MovieCard(). onDetailClick: navigasi ke DetailScreen berdasarkan ID film. onWebsiteClick: membuka situs TMDB dari film yang diklik menggunakan Intent. Baris [121– 126] mendefinisikan fungsi MovieCard() yang menerima data Movie, serta dua lambda (onDetailClick, onWebsiteClick) untuk aksi tombol. Baris [127 – 147] membungkus seluruh kartu film dalam Card dengan padding dan lebar penuh. Kemudian membuat Row horizontal, dan menampilkan gambar poster film menggunakan AsyncImage, dengan pengaturan ukuran, ContentScale.Crop, placeholder dan fallback error.

Baris [149 - 204] membuat Column berisi informasi teks tentang film, meliputi judul film menggunakan titleMedium, overview singkat maksimal 3 baris, dengan TextOverflow.Ellipsis, menampilkan tanggal rilis film, rating film dengan format 1 angka desimal, dua tombol di Row: "Detail" untuk navigasi, dan "TMDB" untuk membuka website resmi film.

DetailScreen.kt

Baris [1–23] mengimpor semua komponen yang diperlukan untuk membangun UI menggunakan Jetpack Compose, Material3, Coil (untuk gambar), serta utilitas navigasi dan handling respons dari API.

Baris [25 – 32] mendefinisikan fungsi `DetailScreen()` yang menampilkan detail film berdasarkan `movieId`, serta menangani navigasi dan pemantauan state dari `viewModel`. Kemudian, baris [34 – 42] menampilkan loading spinner saat data film masih dimuat. Kemudian, Baris [48 – 54] menampilkan data film saat berhasil dimuat. Setelah itu, Baris [55 – 96] menampilkan backdrop dan poster film dengan layout tumpang tindih untuk efek visual menarik.

Baris [97 – 159] menampilkan informasi dasar film seperti judul, tanggal rilis, bahasa, rating, dan jumlah vote. Baris [161 – 181] menampilkan maksimal tiga genre dalam bentuk `AssistChip`. Baris [183 – 207] menampilkan ringkasan film (overview) dengan teks yang dirapikan agar nyaman dibaca. Baris [209 – 236] menampilkan tombol kembali (ikon panah) di pojok kiri atas layar. Baris [238 – 267] menampilkan pesan kesalahan jika terjadi error saat pengambilan data.

AppNavigation.kt

Baris [1 – 14] merupakan bagian impor yang menyertakan komponen-komponen utama dari Jetpack Compose seperti `Scaffold` untuk kerangka layout, padding, serta fungsi-fungsi navigasi seperti `NavHost`, `composable`, dan `rememberNavController`. Juga diimpor `HomeScreen` dan `DetailScreen` sebagai layar yang akan ditampilkan dalam navigasi aplikasi.

Baris [17 – 42] mendefinisikan fungsi `AppNavHost()` sebagai pengatur navigasi utama aplikasi. Di dalamnya dibuat `navController` menggunakan `rememberNavController()` untuk mengelola perpindahan layar. Kemudian `Scaffold` digunakan sebagai struktur UI utama, dan `NavHost` didefinisikan dengan `startDestination` ke `"home_screen"`. Rute `"home_screen"` akan menampilkan `HomeScreen` dan menyertakan `navController` sebagai parameter. Sementara rute `"detail_screen/{movieId}"` menerima parameter `movieId` dari `BackStackEntry`, mengubahnya menjadi `Int`, dan mengirimkannya ke `DetailScreen` bersama fungsi `onBackClick` yang menavigasi kembali ke layar sebelumnya dengan `popBackStack()`. Struktur ini memungkinkan navigasi dinamis antar layar dalam aplikasi.

MovieApplication.kt

Baris [1–3] merupakan bagian impor yang menyertakan `Application` dari `Android` dan anotasi `@HiltAndroidApp` dari `Dagger Hilt`. Ini diperlukan untuk menginisialisasi sistem `dependency injection` di seluruh aplikasi.

Baris [5–7] mendefinisikan kelas `MovieApplication` yang mewarisi dari `Application` dan dianotasi dengan `@HiltAndroidApp`. Anotasi ini memberi tahu `Hilt` untuk menghasilkan kode yang diperlukan agar `dependency injection` dapat bekerja secara global di aplikasi. Dengan ini, `Hilt` akan secara otomatis mengelola siklus hidup dependensi dan menyediakan instance yang dibutuhkan saat aplikasi dijalankan.

MainActivity.kt

Baris [1–7] berisi bagian impor yang mencakup komponen utama `Android` seperti `ComponentActivity`, `Bundle`, dan `setContent`, serta fungsi `enableEdgeToEdge` untuk mendukung tampilan layar penuh. Selain itu, juga diimpor `AppNavHost` untuk navigasi antar layar dan `RetrofitTheme` sebagai tema aplikasi. Anotasi `@AndroidEntryPoint` dari `Hilt` juga diimpor untuk mendukung `dependency injection` pada `MainActivity`.

Baris [9–18] mendefinisikan kelas `MainActivity` yang merupakan `ComponentActivity` dan menjadi titik masuk utama aplikasi. Diberi anotasi `@AndroidEntryPoint` agar dapat menerima dependensi dari `Hilt`. Di dalam fungsi `onCreate`, dipanggil `enableEdgeToEdge()` untuk mengaktifkan tampilan layar penuh tanpa batas, lalu `setContent` digunakan untuk mengatur UI aplikasi menggunakan `Jetpack Compose`. Di dalamnya, diterapkan `RetrofitTheme` sebagai styling global, dan `AppNavHost()` sebagai kontainer navigasi utama yang menangani perpindahan layar seperti `Home` dan `Detail`.

TAUTAN GIT

Berikut adalah tautan untuk source code yang telah dibuat.

https://github.com/rnddfn/Pemrograman_Mobile